

Guide 2: Linux Server Setup (With Router Access)

This guide covers installing Ubuntu Server on an old laptop or desktop, for situations where you have access to your router's admin settings.

Step 1 - Prerequisites

1. Download Ubuntu Server

Go to ubuntu.com/download/server and download the latest LTS (Long Term Support) release. LTS versions get 5 years of security updates, making them the best choice for a server you'll rely on long-term.

2. Create a bootable USB drive

You'll need a spare USB stick (8GB or larger - all data on it will be erased).

Download Rufus (Windows tool for creating bootable USB drives). Then:

- Insert your USB stick
- Open Rufus
- Select the USB drive under "Device"
- Click "Select" and choose the Ubuntu Server ISO you downloaded
- Leave other settings as default
- Click "Start" and wait for it to complete

3. Check the target machine's boot settings

On the laptop/desktop you're installing to:

- Find out the key to access the boot menu (commonly F12, F2, Esc, or Del - varies by manufacturer). This lets you choose to boot from USB for just this one boot, without changing any permanent settings.
- Some older machines have USB booting disabled by default in BIOS/UEFI settings. If the boot menu doesn't show your USB drive as an option, you'll need to go into BIOS settings and enable USB booting first.
- If Secure Boot causes issues (e.g. the installer won't start), it can usually be temporarily disabled in BIOS settings.
- You generally don't need to permanently change the boot order - the boot menu key lets you pick USB for this one install, and the machine will go back to booting normally afterwards.

Step 2 - Installing Ubuntu Server

1. Boot from the USB drive

Insert the USB stick into the target machine, power it on, and press the boot menu key (from Step 1) repeatedly until the boot menu appears. Select the USB drive.

2. Language and keyboard

Choose your language, then your keyboard layout (usually "English (UK)" if you're in the UK).

3. Choose the type of installation

Select "Ubuntu Server" (the default, curated package set) - not "Ubuntu Server (minimized)", which strips things out for unattended/headless automation setups. Leave "Search for third-party drivers" unticked. Select Done.

4. Network connections

The installer will detect available network interfaces (ethernet/WiFi) and attempt to get an IP address via DHCP automatically. Let it use DHCP - we'll set up a reservation for this address on your router in Step 4. If you're using WiFi, you may need to select your network and enter the password here. Select Done once an IP is shown.

5. Proxy and mirror

Leave these as default - the installer automatically tests and confirms a working mirror (you'll see a log showing "passed tests"). Select Done.

6. Storage configuration

For a dedicated server, "Use an entire disk" is the simplest option. Select your target disk - double-check this is correct, as it will erase everything on it. You'll be shown a summary of the partition layout - accept the defaults unless you have specific requirements.

7. Profile setup

Enter your name, choose a server name (hostname) - this is how the server identifies itself on your network, so pick something memorable. Then create a username and password - make sure to remember these, as you'll need them to log in.

8. SSH setup

Tick "Install OpenSSH server" - this lets you connect to your server remotely once it's set up.

9. Featured server snaps

The installer offers to pre-install popular server applications ("snaps") - Docker, Nextcloud, and similar. Skip these for now by pressing Done without selecting anything; you can install anything you need later once the server is running.

10. Installation

The installer will now copy files and configure your system. This takes a few minutes. Once complete, remove the USB drive when prompted and reboot.

11. First login

After reboot, log in with the username and password you created. You're now running Ubuntu Server!

Note: Ubuntu Server has no graphical interface (GUI) - everything is done through the command line (CLI). This is normal and by design; servers don't need a screen/keyboard/mouse once set up, and the CLI uses far fewer resources.

Step 3 - First Boot Checks

After logging in for the first time, it's worth running a few checks to confirm everything's working before making any changes.

1. Check your network interface name and current IP

```
ip a
```

This lists all network interfaces. Look for one with an IP address under inet (e.g. 192.168.x.x). Note the interface name (e.g. enp0s3, eth0) - you'll need this in Step 4.

2. Check you have internet access

```
ping -c 4 google.com
```

The -c 4 sends just 4 pings then stops. You should see replies - if not, double-check your network connection/cable.

3. Update the system

```
sudo apt update && sudo apt upgrade -y
```

This refreshes the package list (update) and installs any available updates (upgrade) - good practice on a fresh install. The -y automatically confirms "yes" to any prompts.

4. Note your hostname

```
hostname
```

This confirms the server name you set during installation - useful for SSH later.

Step 4 - Setting a DHCP Reservation

This is the router-access equivalent of a static IP - instead of configuring the server directly, you tell your router to always assign the same IP to your server's network card. This is much safer, since you're not touching the server's own network config at all.

1. Find your server's MAC address

```
ip a
```

You'll see one or more network interfaces - commonly enp0s3 (ethernet) or wlp0s20f3 (WiFi/wireless, "wlp" prefix). If your server only has WiFi, look for the wlp interface instead. Look for the line starting link/ether under your active interface - this is a unique hardware address written in hexadecimal (base 16, using digits 0-9 and letters a-f), e.g. 08:00:27:4e:eb:1d. Note this down - you'll need it exactly as shown.

2. Log into your router's admin page

This is typically done by entering your router's IP address (often 192.168.1.1 or 192.168.0.1) into a web browser on a device connected to the same network. You'll need your router's admin username/password - often printed on a sticker on the router itself, or check your ISP's documentation if it's been changed.

3. Find the DHCP reservation / static lease section

This is usually under a menu like "LAN Settings", "DHCP Server", or "Network Settings" - the exact wording and location varies significantly between router manufacturers (BT, Sky, Virgin, TP-Link, Netgear, etc.).

Note: This step is for a DHCP reservation only, which keeps your server's IP consistent within your home network. This is different from port forwarding, which allows access from outside your home network (e.g. the internet) to a specific service. Port forwarding isn't needed for this guide - we're only ensuring a stable local IP.

4. Add a reservation

You'll typically need to enter:

- The MAC address of your server (from step 1)
- The IP address you want it to always have - this should be on the same subnet as your other devices (e.g. if your devices are 192.168.1.x, pick something like 192.168.1.50). Many router admin pages show a list of currently connected devices and their IPs nearby - you can use the server's current DHCP-assigned IP (from ip a in Step 3) as the reservation, which is often the simplest choice
- A name/description (e.g. "homeserver")

Save the settings.

Step 5 - Verifying the Reservation Persists

After setting the reservation and rebooting, let's confirm it's working reliably.

1. Check the current IP

```
ip a
```

Confirm it matches the IP you set in the router's reservation.

2. Reboot again and recheck

```
sudo reboot
```

After it comes back up, run ip a again. If the reservation is working correctly, the IP should be identical both times.

3. Optional - power cycle test

For extra confidence, you could also try fully powering off the server (not just rebooting), waiting a minute, then powering back on. DHCP reservations can sometimes behave differently after a longer disconnection versus a quick reboot - though in practice this is rare.

If the IP stayed the same across reboots, your reservation is working - your server now has a stable local IP without any risky network config changes on the server itself.

Step 6 - SSH Access from the Local Network

Why SSH?

SSH (Secure Shell) lets you control your server remotely from another device, using just the command line - no need for a screen, keyboard, or mouse connected to the server itself. This is essential for a headless server (one without a monitor), and is how you'll do almost everything going forward.

Pros

- No physical access needed - manage your server from your main PC, laptop, or even phone, as long as you're on the same network
- Encrypted - traffic between your device and the server is secure, even on shared networks
- Lightweight - just a terminal window, no extra software needed beyond an SSH client (built into Windows, Mac, and Linux)
- Copy/paste friendly - much easier to work with commands, configs, and output than typing directly on a connected keyboard

Cons / Things to Be Aware Of

- Requires the server to be on and connected - if it's powered off or has no network connection, SSH won't work (this is where the static IP from Step 4/5 helps)
- Local network only by default - accessing your server from outside the home (e.g. while out, or on mobile data) requires the additional setup covered in Step 7
- Password-based login by default - while encrypted, using a password means anyone who guesses/obtains it could log in. Switching to key-based SSH authentication is a more secure option, but is a more involved topic covered elsewhere

Connecting via SSH

1. Find your server's IP

```
ip a
```

This is the reserved IP from Step 4/5.

2. From another device on the same network, open a terminal

- Windows: open PowerShell
- Mac/Linux: open Terminal

3. Connect using SSH

```
ssh username@192.168.1.x
```

Replace username with the username you created during installation, and 192.168.1.x with your server's IP.

4. First connection warning

The first time you connect, you'll see a message about the server's authenticity not being verified, asking if you want to continue. This is normal for a first connection - type yes and press Enter.

5. Enter your password

Type the password you set during installation (it won't show as you type - this is normal) and press Enter.

You should now see your server's command prompt - you're connected!

Step 7 - Optional: Remote Access Away From Home

Everything so far covers access within your home network. If you'd also like to access your server when you're away from home - for example, checking on it from your phone while out, or connecting from a laptop at work - you'll need a way to reach your server from outside your home network.

This is entirely optional - if you only ever need access while at home, you can skip this step.

The Challenge

Your home network is private - devices outside it can't normally "see" your server, even though it has an IP address (this is by design, for security). To access it remotely, you need either:

- Port forwarding - configuring your router to forward specific connections from the internet to your server. This works, but opens up your home network to the internet, which carries security risks if not configured carefully

- A VPN-style service like Tailscale - creates a private, encrypted connection between your devices without exposing anything to the wider internet

Tailscale (Recommended)

Tailscale is a free service that gives your server (and your other devices - phone, laptop, etc.) a permanent private address in the 100.x.x.x range. This address works from anywhere with an internet connection - including WiFi, and mobile data (3G/4G/5G or equivalent) - without needing to configure your router at all.

1. Install Tailscale on your server

```
curl -fsSL https://tailscale.com/install.sh | sh
```

2. Start Tailscale and authenticate

```
sudo tailscale up
```

This will output a URL - open it in a browser (on any device) and log in with a Tailscale account (free - sign in with Google, Microsoft, or GitHub). You'll see a "Connect device" confirmation page - click Connect to authorize this machine. The terminal command should then complete automatically.

Note: If the command appears to hang after opening the link, check the Tailscale admin console (login.tailscale.com) - the device may show as connected there even if the terminal hasn't updated. A reboot (sudo reboot) will usually resync the connection.

3. Find your server's Tailscale IP

```
tailscale ip -4
```

This shows the permanent 100.x.x.x address for this server.

4. Install Tailscale on your other devices

Install the Tailscale app on any device you want to connect from - phone (iOS/Android), laptop (Windows/Mac/Linux), etc. - available at tailscale.com/download. Sign in with the same account.

5. Connect remotely

From a Tailscale-connected device, away from home:

```
ssh username@100.x.x.x
```

(Replace with your server's Tailscale IP and your username.) On mobile, an SSH client app (e.g. Termius, JuiceSSH) lets you do this from your phone too.

Guide 2: Linux Server Setup (With Router Access) - Summary

Goal: Get a self-hosted Linux server up, running, and reliably accessible on your home network - using your router's settings to keep things simple and low-risk.

What this guide covers:

- Prerequisites - Downloading Ubuntu Server LTS, creating a bootable USB with Rufus, and checking boot settings on the target machine.
- Installation - A full walkthrough of the Ubuntu Server installer, including base installation type, network setup (DHCP), storage, profile/hostname setup, and enabling OpenSSH.

- First Boot Checks - Confirming network connectivity, checking your IP/interface name, updating the system, and noting your hostname.
- DHCP Reservation - Using your router's admin settings to ensure your server always gets the same local IP, based on its MAC address - a safer alternative to configuring a static IP directly on the server.
- Verifying the Reservation - Confirming the IP stays consistent across reboots.
- SSH Access - Connecting to your server from another device on the same network, enabling full headless management.
- Optional - Remote Access Away From Home - Installing Tailscale for secure access to your server from anywhere with an internet connection, including mobile data.

Key takeaway: With router access, getting a stable, reliable local IP is straightforward and low-risk via a DHCP reservation - no need to edit network config files on the server itself. Combined with SSH (and optionally Tailscale for remote access), this gives you a fully manageable headless server.